

**SIMATS ENGINEERING
ASSESSMENT TOOL 1**

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO2: Experiment search and game-solving techniques such as Greedy Search, A*, CSP, Minimax, and Alpha-Beta pruning with heuristic functions for solving complex AI problems efficiently. (BL3)
Assessment Tool Weightage: 50%

QUESTIONS: 20

**Duration : 30 Minutes
Total Marks:20**

Annexure A
MCQ Test

Code of Conduct:

I B Sandhya(192311292) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.
I understand that any violation of academic integrity rules will result in disciplinary action.
Signature of the student: Sandhya

Annexure A
MCQ Test

1.In informed search strategies, some algorithms use only the heuristic value to decide which node to expand next. This approach focuses on selecting the node that appears closest to the goal without considering the cost already incurred.

Which algorithm follows this principle?

- A) Breadth First Search
- B) Uniform Cost Search
- C) Greedy Best-First Search
- D) Depth First Search

ANSWER

The correct answer is:

☒ **C) Greedy Best-First Search**

Explanation:

- **Greedy Best-First Search (GBFS)** selects the next node based **only on the heuristic value $h(n)$** .
- It always expands the node that **appears closest to the goal**, without considering the path cost already traveled **$g(n)$** .

Evaluation Function:

$$f(n)=h(n)$$

Why the other options are incorrect:

- **A) Breadth First Search (BFS):** Explores nodes level by level; it does **not** use a heuristic.
- **B) Uniform Cost Search (UCS):** Uses only the path cost **$g(n)$** , not the heuristic.
- **D) Depth First Search (DFS):** Explores as deep as possible first; it does **not** use a heuristic.

✓ Final Answer: C) Greedy Best-First Search

2. A* search is widely used in pathfinding and graph traversal because it combines the actual cost from the start node and the estimated cost to the goal. This balance allows it to find optimal solutions efficiently when the heuristic is admissible.

What is the evaluation function used in A* search?

- A) $f(n) = g(n)$
- B) $f(n) = h(n)$
- C) $f(n) = g(n) + h(n)$
- D) $f(n) = g(n) \times h(n)$

ANSWER

The correct answer is:

✓ C) $f(n)=g(n)+h(n)$

Explanation:

A* Search uses both:

$g(n)$: Actual cost from the start node to the current node.

$h(n)$: Estimated (heuristic) cost from the current node to the goal.

The evaluation function is:

$$f(n)=g(n)+h(n)$$

This combination helps A* find the **optimal path** efficiently when the heuristic is **admissible** (never overestimates the true cost).

Why the other options are incorrect:

- A) $f(n)=g(n)$:** Used by **Uniform Cost Search**.
- B) $f(n)=h(n)$:** Used by **Greedy Best-First Search**.
- D) $f(n)=g(n)\times h(n)$:** Not used in the A* algorithm.

✓ **Final Answer: C) $f(n)=g(n)+h(n)$**

3. Heuristic functions play a crucial role in informed search by guiding the algorithm toward the goal. A good heuristic should not overestimate the actual cost and should improve search efficiency.

What property ensures optimality in A* search?

- A) Completeness
- B) Admissibility
- C) Depth limit
- D) Backtracking

Answer: B

ANSWER

The correct answer is:

✓ **B) Admissibility**

Explanation:

A heuristic is **admissible** if it **never overestimates** the actual minimum cost to reach the goal.

When A* uses an admissible heuristic, it is guaranteed to find the **optimal (shortest) path**.

Why the other options are incorrect:

A) Completeness: Ensures the algorithm finds a solution if one exists, but does not guarantee it is optimal.

C) Depth limit: Used in Depth-Limited Search, not related to A* optimality.

D) Backtracking: Used in constraint satisfaction and search problems, not for ensuring A* optimality.

✓ **Final Answer: B) Admissibility**

4. Local search algorithms operate by starting with an initial solution and iteratively improving it based on a given objective function. These methods are commonly used in optimization problems where the path is not important but the final solution is.

Which of the following is a local search algorithm?

- A) A* Search
- B) Hill Climbing
- C) Breadth First Search
- D) Uniform Cost Search

ANSWER

The correct answer is:

✓ **B) Hill Climbing**

Explanation:

Hill Climbing is a **local search algorithm**.

It starts with an initial solution and repeatedly moves to a neighboring solution with a better objective (heuristic) value until no better neighbor exists.

Why the other options are incorrect:

A) A Search:* An informed search algorithm for finding the shortest path.

C) **Breadth First Search (BFS)**: An uninformed search algorithm that explores level by level.

D) **Uniform Cost Search (UCS)**: An uninformed search algorithm that expands the node with the lowest path cost.

✓ **Final Answer: B) Hill Climbing**

5. In adversarial search, agents compete against each other in a game environment. The decision-making process involves considering both the agent's moves and the opponent's possible responses.

Which algorithm is commonly used for such decision-making?

- A) Greedy Search
- B) Minimax Algorithm
- C) Backtracking
- D) Uniform Search

ANSWER

The correct answer is:

✓ **B) Minimax Algorithm**

Explanation:

The **Minimax Algorithm** is used in **adversarial search** (game-playing AI).

It assumes two players:

MAX tries to maximize the score.

MIN tries to minimize the score.

The algorithm chooses the move that gives the best possible outcome while assuming the opponent also plays optimally.

Why the other options are incorrect:

A) Greedy Search: Chooses the locally best option without considering an opponent.

C) Backtracking: Used for constraint satisfaction problems, not adversarial games.

D) Uniform Search (Uniform Cost Search): Finds the least-cost path and is not used for game decision-making.

✓ **Final Answer: B) Minimax Algorithm**

6. A delivery robot in a warehouse selects paths based only on the estimated distance to the destination without considering obstacles already encountered. This sometimes leads to suboptimal paths but faster decisions.

Which algorithm is the robot using?

A) A* Search

B) Greedy Best-First Search

C) Uniform Cost Search

D) Depth Limited Search

ANSWER

The correct answer is:

✓ **B) Greedy Best-First Search**

Explanation:

Greedy Best-First Search (GBFS) selects the next node **only based on the heuristic estimate** $h(n)$, such as the estimated distance to the destination.

It **does not consider the actual path cost** or obstacles already encountered, so it may find **suboptimal paths**, but it often makes decisions quickly.

Why the other options are incorrect:

A) A Search: * Uses both the actual cost and heuristic $f(n)=g(n)+h(n)$, making it more likely to find the optimal path.

C) **Uniform Cost Search (UCS):** Considers only the actual path cost $g(n)$, not the heuristic.

D) **Depth Limited Search:** Explores up to a specified depth and does not use heuristic estimates.

✓ **Final Answer: B) Greedy Best-First Search**

7. A GPS navigation system calculates routes by combining the distance already travelled and an estimate of the remaining distance to the destination. This helps in finding the shortest path efficiently.

Which algorithm best represents this approach?

A) Depth First Search

- B) A* Search
- C) Hill Climbing
- D) Backtracking

ANSWER

The correct answer is:

- ☒ B) A Search*

Explanation:

A* Search combines:

g(n): Actual distance (cost) already traveled from the start.

h(n): Estimated remaining distance (heuristic) to the destination.

Its evaluation function is:

$$f(n)=g(n)+h(n)$$

This allows A* to efficiently find the **shortest (optimal) path** when the heuristic is admissible.

Why the other options are incorrect:

A) Depth First Search (DFS): Explores one path deeply without considering path cost or heuristics.

C) Hill Climbing: Uses only the heuristic and may get stuck in local optima.

D) Backtracking: Used for constraint satisfaction problems, not shortest-path navigation.

- ☒ Final Answer: B) A Search*

8. A university is scheduling exams for multiple subjects with constraints such as no overlapping exams and limited rooms. The solution must satisfy all constraints while assigning time slots.

This problem is best modeled as:

- A) Game Playing Problem
- B) Constraint Satisfaction Problem
- C) Heuristic Search Problem
- D) Optimization Problem

ANSWER

The correct answer is:

- ☒ B) Constraint Satisfaction Problem

Explanation:

A **Constraint Satisfaction Problem (CSP)** involves assigning values to variables while satisfying a set of constraints.

In exam scheduling:

Variables: Exams

Values: Time slots and rooms

Constraints:

No overlapping exams for the same students.

Limited number of rooms.

One exam per room at a time.

The goal is to find an assignment that satisfies **all constraints**.

Why the other options are incorrect:

A) Game Playing Problem: Involves competing players (e.g., chess).

C) Heuristic Search Problem: Uses heuristics to guide search but is not specifically about satisfying constraints.

D) Optimization Problem: Focuses on finding the best solution, while this question emphasizes satisfying all constraints.

✓ Final Answer: B) Constraint Satisfaction Problem

9. An AI-based chess program evaluates all possible moves and anticipates the opponent's responses before making a decision. It assigns utility values to game states to determine the best move.

Which algorithm is being applied here?

- A) Greedy Search
- B) Hill Climbing
- C) Minimax Algorithm
- D) Breadth First Search

ANSWER

The correct answer is:

✓ C) Minimax Algorithm**Explanation:**

The **Minimax Algorithm** is used in **AI game playing** (such as chess).

It evaluates possible moves by considering:

The AI's best move (**MAX** player).

The opponent's best possible response (**MIN** player).

It assigns **utility values** to game states and chooses the move that maximizes the AI's chances of winning while minimizing the opponent's advantage.

Why the other options are incorrect:

A) Greedy Search: Chooses the locally best move without considering future opponent moves.

B) Hill Climbing: A local search algorithm that does not model adversarial opponents.

D) Breadth First Search (BFS): Explores nodes level by level and is not designed for adversarial game decision-making.

☒ **Final Answer: C) Minimax Algorithm**

10. A robot explores an unknown environment where it does not have prior knowledge of obstacles. It makes decisions step-by-step and updates its knowledge as it moves. Which type of agent is suitable for this scenario?

- A) Offline Agent
- B) Online Search Agent
- C) Static Agent
- D) Deterministic Agent

ANSWER

The correct answer is:

☒ **B) Online Search Agent**

Explanation:

An **Online Search Agent** operates in an **unknown environment**.

It **does not have prior knowledge** of obstacles or the map.

The agent **explores, gathers information, and updates its knowledge step-by-step** while making decisions.

Why the other options are incorrect:

A) Offline Agent: Requires complete knowledge of the environment before searching.

C) Static Agent: Assumes the environment does not change and does not actively learn while moving.

D) Deterministic Agent: Refers to environments where actions have predictable outcomes, not to learning unknown environments.

✓ **Final Answer: B) Online Search Agent**

11. A* search guarantees optimal solutions only when certain conditions are satisfied by the heuristic function. If these conditions are violated, the algorithm may still find a solution but not necessarily the best one.

Why is an admissible heuristic important?

- A) It reduces memory usage
- B) It ensures optimality
- C) It increases branching factor
- D) It avoids loops

ANSWER

The correct answer is:

✓ **B) It ensures optimality**

Explanation:

An **admissible heuristic** never **overestimates** the true cost to reach the goal.

In **A*** Search, using an admissible heuristic guarantees that the algorithm finds the **optimal (shortest) solution**.

Why the other options are incorrect:

A) It reduces memory usage: Memory usage depends on the search algorithm, not on heuristic admissibility.

C) It increases branching factor: The branching factor is determined by the problem, not the heuristic.

D) It avoids loops: Loop avoidance is handled by tracking visited states, not by heuristic admissibility.

✓ **Final Answer: B) It ensures optimality**

12. In large game trees, evaluating every node becomes computationally expensive. Alpha-Beta pruning helps eliminate branches that do not affect the final decision, thereby improving efficiency.

What is the primary benefit of Alpha-Beta pruning?

- A) Reduces depth of tree
- B) Reduces number of nodes evaluated

- C) Changes game rules
- D) Improves heuristic accuracy

ANSWER

The correct answer is:

☒ **B) Reduces number of nodes evaluated**

Explanation:

Alpha-Beta Pruning is an optimization of the **Minimax Algorithm**.

It **prunes (skips)** branches that cannot influence the final decision.

This reduces the **number of nodes evaluated**, making the search much faster while still producing the same optimal move as Minimax.

Why the other options are incorrect:

A) Reduces depth of tree: It does not change the depth of the game tree.

C) Changes game rules: It only optimizes the search process; it does not affect the game rules.

D) Improves heuristic accuracy: Alpha-Beta pruning does not modify or improve heuristics.

☒ **Final Answer: B) Reduces number of nodes evaluated**

13. Hill Climbing algorithm continuously moves toward better states based on local improvements. However, it may fail to reach the global optimum due to certain limitations in the search space.

What is the main drawback of Hill Climbing?

- A) High memory usage
- B) Revisiting nodes
- C) Getting stuck in local maxima
- D) Requires complete tree

Answer: C

ANSWER

The correct answer is:

☒ **C) Getting stuck in local maxima**

Explanation:

Hill Climbing always moves to the neighboring state with the best improvement.

It can become **stuck at a local maximum (or local optimum)**, where no neighboring state is better, even though a better global solution exists.

Why the other options are incorrect:

A) High memory usage: Hill Climbing uses very little memory.

B) Revisiting nodes: Revisiting nodes is not its primary drawback.

D) Requires complete tree: Hill Climbing does not require a complete search tree.

✓ **Final Answer: C) Getting stuck in local maxima**

14. Constraint Satisfaction Problems involve variables, domains, and constraints. A valid solution must assign values to all variables such that no constraints are violated.

What ensures a valid CSP solution?

A) Heuristic function

B) Path cost

C) Constraint satisfaction

D) Depth-first traversal

ANSWER

The correct answer is:

✓ **C) Constraint satisfaction**

Explanation:

In a **Constraint Satisfaction Problem (CSP)**, a solution is valid only if **all variables are assigned values** and **all constraints are satisfied**.

If even one constraint is violated, the assignment is **not** a valid solution.

Why the other options are incorrect:

A) Heuristic function: Helps guide search but does not guarantee a valid CSP solution.

B) Path cost: Relevant to pathfinding algorithms, not CSPs.

D) Depth-first traversal: A search strategy that does not by itself ensure all constraints are satisfied.

✓ **Final Answer: C) Constraint satisfaction**

15. In adversarial games, evaluation functions are used when it is not feasible to explore the entire game tree. These functions estimate the desirability of a game state.

What is the role of an evaluation function?

A) Generate successors

B) Estimate utility of a state

- C) Reduce branching factor
- D) Store moves

ANSWER

The correct answer is:

✓ **B) Estimate utility of a state**

Explanation:

An **evaluation function** estimates how good or bad a game state is when searching the entire game tree is not feasible.

It assigns a **utility (score)** to each state, helping algorithms like **Minimax** choose the best move.

Why the other options are incorrect:

A) Generate successors: Successor functions generate possible next states; evaluation functions do not.

C) Reduce branching factor: Evaluation functions do not reduce the number of possible moves.

D) Store moves: Storing moves is not the purpose of an evaluation function.

✓ **Final Answer: B) Estimate utility of a state**

16. In A* search, the evaluation function is calculated using both the path cost and heuristic value. Suppose a node has a path cost of 7 and a heuristic estimate of 5. What is the value of $f(n)$?

- A) 12
- B) 2
- C) 35
- D) 7

ANSWER

The correct answer is:

✓ **A) 12**

Explanation:

The evaluation function in **A*** Search is:

$$f(n) = g(n) + h(n)$$

Where:

$g(n) = 7$ (path cost from the start)

$h(n) = 5$ (heuristic estimate to the goal)

So,

$$f(n) = 7 + 5 = 12$$

Why the other options are incorrect:

B) 2: Incorrect calculation.

C) 35: This is 7×5 , but A* uses addition, not multiplication.

D) 7: This considers only the path cost and ignores the heuristic.

☒ **Final Answer: A) 12**

17. A heuristic function sometimes overestimates the actual cost to reach the goal. This violates the admissibility condition required for optimal solutions in A* search.

What happens in such a case?

A) A* remains optimal

B) A* becomes non-optimal

C) A* becomes BFS

D) A* stops working

ANSWER

The correct answer is:

☒ B) A becomes non-optimal*

Explanation:

If the heuristic **overestimates** the actual cost, it is **not admissible**.

In this case, A* may still find a solution, but it is **not guaranteed to find the optimal (shortest) path**.

Why the other options are incorrect:

A) A remains optimal:* Incorrect, because optimality is no longer guaranteed.

C) A becomes BFS:* A* does not change into Breadth-First Search.

D) A stops working:* A* still works and can find a solution; it just may not be the best one.

☒ **Final Answer: B) A becomes non-optimal***

18. Consider a search tree with a branching factor of 2 and depth of 4. The total number

of nodes generated in a full binary tree can be calculated analytically.
What is the approximate number of nodes?

- A) 10
- B) 15
- C) 31
- D) 16

ANSWER

The correct answer is:

☒ C) 31

Explanation:

For a **full binary tree** with:

- **Branching factor (b) = 2**
- **Depth (d) = 4**

The total number of nodes is:

$$1+2+2^2+2^3+2^4 = 1+2+4+8+16=31$$

Alternatively, using the formula:

$$\text{Total Nodes} = b^{d+1} - 1$$

Substituting $b=2$ and $d=4$:

$$2^{4+1} - 1 = 2^5 - 1 = 32 - 1 = 31$$

Why the other options are incorrect:

- **A) 10:** Too few nodes.
- **B) 15:** This is the total number of nodes up to depth 3.
- **D) 16:** This is only the number of nodes at depth 4, not the total.

☒ **Final Answer: C) 31**

19. In a minimax tree, each node has 3 children and the depth of the tree is 3. The total number of leaf nodes depends on the branching factor raised to the depth.

How many leaf nodes are present?

- A) 9
- B) 27
- C) 6
- D) 12

ANSWER

The correct answer is:

✓ B) 27

Explanation:

The number of **leaf nodes** in a tree is calculated as:

Leaf Nodes = b^d

Where:

- **Branching factor (b) = 3**
- **Depth (d) = 3**

So,

$3^3 = 27$

Therefore, the tree has **27 leaf nodes**.

Why the other options are incorrect:

- **A) 9:** 32, incorrect depth.
- **C) 6:** Incorrect calculation.
- **D) 12:** Does not match the formula.

✓ **Final Answer: B) 27**

20. Alpha-Beta pruning improves the efficiency of minimax by reducing the number of nodes evaluated. In the best-case scenario, it significantly lowers time complexity.

What is the best-case time complexity?

- A) $O(b^d)$
- B) $O(b^{(d/2)})$
- C) $O(d^b)$
- D) $O(\log b)$

ANSWER

The correct answer is:

✓ B) $O(b^{(d/2)})$

Explanation:

- The **Minimax algorithm** has a time complexity of:

$O(b^d)$

where:

- **b** = branching factor

- **d** = depth of the game tree
- With **Alpha-Beta Pruning**, in the **best case** (perfect move ordering), the number of nodes evaluated is greatly reduced.

The **best-case time complexity** becomes:

$$O(b^{d/2})$$

This allows the algorithm to search approximately **twice as deep** in the same amount of time compared to standard Minimax.

Why the other options are incorrect:

- **A) $O(b^d)$:** This is the time complexity of standard Minimax without pruning.
- **C) $O(d^b)$:** Incorrect complexity expression.
- **D) $O(\log b)$:** Far too low; Alpha-Beta pruning does not achieve logarithmic complexity.

✓ **Final Answer: B) $O(b^{d/2})$**